

Arquitectura Transformer y Mecanismo de Atención en Ciberseguridad

En esta sesión, exploraremos la arquitectura de los **Transformers**, un avance fundamental en el *Deep Learning* que ha revolucionado el procesamiento de datos secuenciales. Aunque la arquitectura es compleja, nos centraremos en comprender cómo funcionan sus componentes clave y sus aplicaciones, particularmente el mecanismo de **Atención**, que es la base de su éxito.

1. El Mecanismo de Atención y Autoatención (*Self-Attention*)

La parte principal del diseño del Transformer es el **mecanismo de atención**, que permite al modelo enfocarse en diferentes partes de la secuencia de entrada, asignando niveles variables de importancia a distintos elementos. Esta capacidad de atención adaptativa es fundamental para que los Transformers capturen relaciones complejas y **dependencias a largo plazo** en datos secuenciales.

A. Conceptos Centrales

La autoatención (*self-attention*) es un componente esencial que permite al modelo evaluar la **importancia relativa** de diferentes partes de una secuencia de entrada. Se basa en tres conceptos clave para cada elemento de la secuencia:

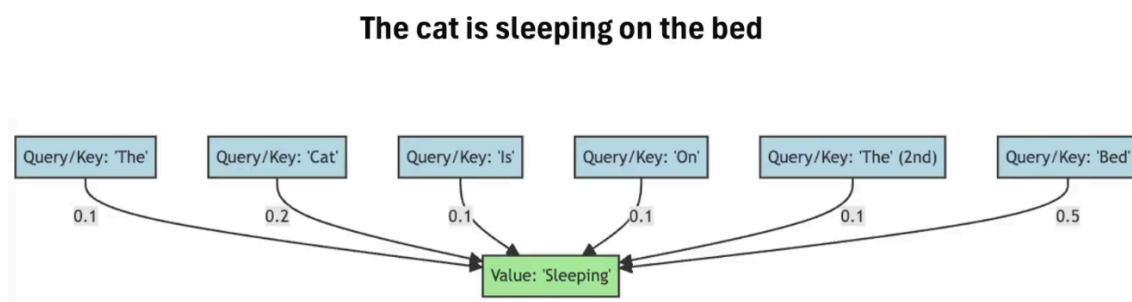
- **Consulta (*Query*)**: Representa la posición actual que se está evaluando o la "pregunta" que el modelo está intentando entender.
- **Clave (*Key*)**: Representa la información descriptiva asociada con todas las posiciones de la secuencia.
- **Valor (*Value*)**: Representa el contenido de información real de la palabra, que se suma y se pondera para crear la salida final.

B. Proceso de Ponderación (*Weighted Sum*)

El proceso se lleva a cabo de la siguiente manera:

1. **Cálculo de Compatibilidad:** Al calcular la compatibilidad entre la **Consulta** y todas sus **Claves**, se obtienen los **pesos de atención**.
2. **Asignación de Importancia:** Estos pesos de atención indican cuánta importancia se le debe dar a cada posición de la secuencia en función de la similitud con la Consulta.
3. **Suma Ponderada (*Weighted Sum*):** Los pesos se utilizan luego para **ponderar los Valores** correspondientes y calcular una **representación contextualizada** de la posición actual.
4. **Vector de Contexto:** El resultado es un vector de contexto para cada palabra que es rico en información contextual, reflejando tanto la significancia de cada palabra como su relación con cada otra palabra de la secuencia. Este vector se convierte en el *output* del *self-attention* y se pasa a la siguiente capa del modelo.

La visualización de estos pesos de atención proporciona una comprensión intuitiva de cómo el modelo asigna importancia a diferentes partes de la secuencia en función del contexto dado, lo que facilita la interpretación de su funcionamiento interno.



El diagrama anterior ilustra este Mecanismo de Atención. En el ejemplo de la frase "The cat is sleeping on the bed", el objetivo es calcular el *Value* para la palabra 'Sleeping'. El mecanismo compara la *Query* de 'Sleeping' con la *Key* de cada palabra en la frase.

- **Alto Peso de Relevancia:** La palabra 'Bed' recibe el peso más alto (0.5), ya que la ubicación es la información más crítica para la acción de "dormir". La palabra 'Cat' (la entidad que realiza la acción) también recibe un peso alto (0.2).
- **Bajo Peso:** Palabras funcionales como 'The', 'Is', y 'On' reciben pesos bajos (0.1) porque no son cruciales para entender la acción.

Finalmente, la nueva representación de 'Sleeping' se calcula sumando los vectores *Value* de todas las demás palabras, cada uno multiplicado por su respectivo peso de atención. Esto permite que la representación final de 'Sleeping' esté fuertemente influenciada por su contexto más relevante ('Bed').



Este otro gráfico muestra los pasos fundamentales del mecanismo de *self-attention*, el cual comienza con la transformación de cada palabra en tres vectores (Query, Key y Value) a través de matrices de pesos aprendibles específicas. La *Query* se utiliza para calcular los *scores* de atención con las *Keys* de todas las palabras, determinando su importancia relativa. Estos *scores* se aplican a los *Values* en un proceso llamado **suma ponderada** (*weighted sum*), donde los vectores *Value* son ponderados de acuerdo con los *scores* de atención calculados, lo que permite a las palabras más relevantes tener más influencia en la representación final. El resultado es un vector de contexto rico que se convierte en el *output* del *self-attention* y se pasa a la siguiente capa.

C. Implicaciones en Ciberseguridad

El Mecanismo de Atención permite que el modelo se enfoque en cualquier parte de la entrada que sea más informativa, en lugar de procesar los datos de forma lineal como en las redes neuronales recurrentes (RNNs) o LSTM.

- **Detección de *Malware*:** En el análisis de código o binarios (que son secuencias de datos), un Transformer puede enfocarse en secuencias clave que indican una carga útil maliciosa (por ejemplo, la relación entre una API de

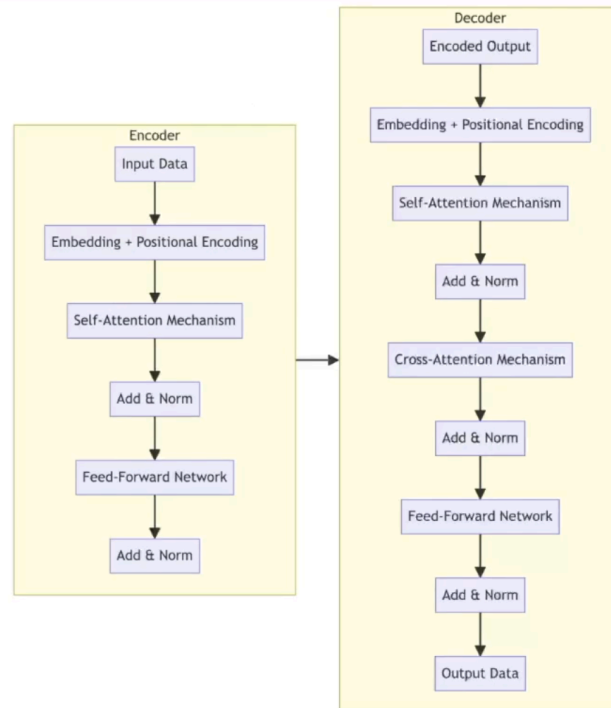
red y una de *shell*), ignorando comandos de inicialización o bucles comunes, sin importar cuán separadas estén en el código.

- **Análisis de Registros de Seguridad (Logs):** En una secuencia de eventos de un *firewall* o un sistema de detección de intrusos (IDS), el modelo puede dar un alto peso de atención a eventos críticos ocurridos hace horas o días (por ejemplo, un inicio de sesión fallido) que son relevantes para el evento actual (por ejemplo, una escalada de privilegios).
 - **Detección de *Phishing* o Fraude (Basada en Texto):** Permite que el modelo identifique relaciones críticas en un correo electrónico de *phishing* (por ejemplo, el texto del cuerpo que sugiere "urgencia") y su relación con la URL del remitente.
-

2. Arquitectura General del Transformer (Encoder-Decoder)

El gráfico de la **Arquitectura del Transformer** describe un diseño fundamental basado en un flujo de **codificador-decodificador** (*Encoder-Decoder*) que sustituye las redes neuronales recurrentes (RNNs) por el mecanismo de atención

Transformer Achitecture
Picture source: own creation



A. El Bloque Codificador (*Encoder*)

El codificador es responsable de procesar la secuencia de entrada (*Input Data*) extrayendo características importantes y creando una representación contextual rica, conocida como el *Encoded Output*.

1. **Embedding y Codificación Posicional:** La *Input Data* se transforma en *embeddings* (representaciones vectoriales). A estos *embeddings* se les añade la **Codificación Posicional** (*Positional Encoding*). Esto es crucial porque el Transformer procesa toda la secuencia en paralelo y necesita un mecanismo para inyectar información sobre la **posición u orden** de los datos en la secuencia.
2. **Mecanismo de Auto-Atención (*Self-Attention*):** Permite que cada elemento de la secuencia de entrada (ej., cada palabra) se compare con todos los demás elementos de la misma secuencia, calculando un peso de atención para entender el contexto.
3. **Adición y Normalización (*Add & Norm*):** La salida del *Self-Attention* se suma al *input* de la capa (una conexión residual, "Add"). Luego, esta suma se

pasa a una capa de

Normalización de Capa (*Layer Normalization*, "Norm"), lo que ayuda a estabilizar el entrenamiento de redes profundas.

4. **Red de Avance (*Feed-Forward Network*)**: Una red neuronal totalmente conectada (*Feed-Forward*) aplica una transformación idéntica a cada posición del vector de forma independiente, introduciendo **no-linealidad**.
5. **Resultado**: El flujo culmina en el **Encoded Output**, que contiene la información contextual de la secuencia de entrada lista para ser utilizada por el decodificador.

B. El Bloque Decodificador (*Decoder*)

El decodificador toma el *Encoded Output* y genera la secuencia de salida (*Output Data*), a menudo paso a paso, como una predicción o traducción.

1. **Embedding y Codificación Posicional**: Al igual que el codificador, comienza con el *Embedding* y la *Positional Encoding* de la secuencia de salida inicial.
2. **Mecanismo de Auto-Atención (Enmascarado)**: Esta capa utiliza una forma **enmascarada** de *Self-Attention*. Esto significa que, al predecir una posición, el modelo solo puede atender a las posiciones que ya han sido generadas, **no a las futuras**, manteniendo así la causalidad y el flujo natural de generación de texto.
3. **Mecanismo de Atención Cruzada (*Cross-Attention*)**: Esta es la capa que **conecta el Codificador y el Decodificador**. La *Query* proviene de la capa de *Self-Attention* del Decodificador (la información que se ha generado hasta ahora), y la *Key* y el *Value* provienen del *Encoded Output* del Codificador. Esto permite al Decodificador enfocarse en las partes más relevantes de la secuencia de entrada para generar la siguiente parte de la secuencia de salida.
4. **Red de Avance (*Feed-Forward Network*)**: Al igual que en el codificador, se aplica una red *Feed-Forward* seguida de otra capa de *Add & Norm*.

El flujo culmina en la *Output Data*, que es la secuencia de salida final generada por el modelo.

3. Codificación Posicional y Entrenamiento

A. Codificación Posicional

La **posición de las palabras** dentro de una secuencia es fundamental para comprender el significado del texto, ya que palabras idénticas en diferentes posiciones pueden tener significados completamente distintos. Por lo tanto, es esencial proporcionar al modelo información sobre la posición de las palabras para capturar su significado en el contexto.

Existen dos enfoques principales para la codificación posicional:

- **Codificación de Posición Absoluta:** Asigna un **vector de posición único** a **cada token** en la secuencia. Este vector codifica la posición exacta del *token* y se aprende junto con los *embeddings* de palabras durante el entrenamiento.
- **Codificación de Posición Relativa:** Captura las **relaciones de distancia entre los tokens**. En lugar de asignar un vector único, utiliza la diferencia entre los índices de posición para codificar la información posicional relativa de un *token* con respecto a otros.

La **codificación posicional se combina con los *embeddings* de palabras** para proporcionar al modelo información tanto sobre el significado semántico como sobre la estructura secuencial del texto, permitiendo una captura efectiva del significado y del contexto.

B. Entrenamiento y Optimización

En las tareas de procesamiento del lenguaje natural, el entrenamiento de modelos Transformer persigue dos objetivos:

- **Modelado del Lenguaje Enmascarado (MLM):** Una parte de las palabras de la secuencia de entrada se **enmascara**, y el modelo debe predecirlas en función del contexto proporcionado por las palabras circundantes. Esto fomenta una comprensión profunda del contexto y una representación robusta de las palabras.

- **Aprendizaje de Secuencia a Secuencia (Seq-to-Seq):** El modelo aprende a **mapear secuencias de entrada a secuencias de salida**, lo cual es esencial para tareas como la traducción automática y la generación de texto.

Los Transformers suelen entrenarse utilizando **optimizadores tipo ADAM**, el cual adapta automáticamente la tasa de aprendizaje para cada parámetro, lo que permite un entrenamiento más eficiente y rápido. Además, se emplea la **programación de tasas de aprendizaje** para ajustar dinámicamente la velocidad de aprendizaje durante el entrenamiento, reduciendo la tasa a medida que el entrenamiento progresa para evitar oscilaciones.

4. Conclusiones y Aplicaciones Generativas

Los Transformers son una arquitectura revolucionaria, especialmente en el ámbito de los **modelos de lenguaje generativo** y los **modelos de lenguaje a gran escala (LLMs)**.

- **Capacidades Clave:** Su capacidad para capturar **relaciones complejas y dependencias a largo plazo** en secuencias de datos, junto a la capacidad de modelar eficazmente la información posicional, los hace extremadamente efectivos en una amplia gama de tareas de NLP.
- **IA Generativa:** Los Transformers son una herramienta invaluable para la **generación de texto, imágenes y otros tipos de contenido creativo**. Han superado a los enfoques tradicionales gracias a su capacidad para modelar dependencias y generar texto con contexto y coherencia, lo que se ve en el éxito de modelos como ChatGPT o Mistral.
- **Visión por Ordenador:** También son muy eficaces en tareas de visión por ordenador, como la **descripción de imágenes**, donde pueden generar descripciones precisas y detalladas basadas en el contenido visual.

Los Transformers han transformado radicalmente el panorama del *Deep Learning* y del procesamiento del lenguaje natural, y su importancia en la IA generativa y en los LLMs será fundamental en los avances futuros en este campo.

Aplicación Práctica: Detección de URLs

Maliciosas con la Arquitectura Transformer y

Hugging Face

En esta sesión, realizaremos un ejercicio práctico crucial para la ciberseguridad: la creación de una aplicación capaz de detectar **URL maliciosas** utilizando la arquitectura **Transformer**. Este ejercicio se simplifica enormemente aprovechando **Hugging Face**, una plataforma que proporciona acceso a modelos de *Machine Learning* (ML) preentrenados a través de una API de inferencia.

1. Hugging Face: Plataforma de Modelos Transformer

Hugging Face es una empresa de tecnología que se ha destacado en el campo de la IA, particularmente en el **Procesamiento del Lenguaje Natural (NLP)**. Es ampliamente conocida por su trabajo con modelos basados en la arquitectura de **Transformers**, que han revolucionado la forma en que las máquinas entienden y generan el lenguaje humano.

La plataforma ofrece una amplia biblioteca de **modelos preentrenados** (como BERT y DistilBERT) y **datasets**, junto con herramientas para el entrenamiento, evaluación y despliegue de modelos de IA. Su popularidad se debe a la **facilidad** con la que investigadores y desarrolladores pueden acceder a tecnología punta, facilitando la experimentación y la implementación de soluciones innovadoras, promoviendo además una comunidad abierta y colaborativa donde los usuarios comparten sus hallazgos y modelos. Esta comunidad es clave para la investigación y el desarrollo de la IA.

Utilizaremos su **API** para conectarnos a un modelo preentrenado mediante *tokens* de autenticación.

2. Procedimiento Práctico: Clasificación de URLs a través de API

El ejercicio demuestra el flujo completo para clasificar URL, desde la carga de un *dataset* local hasta la obtención de predicciones del modelo Transformer alojado remotamente.

Bloque de código entero que vamos a analizar:

```
import requests
import pandas as pd
from sklearn.model_selection import train_test_split

# Load the dataset
df = pd.read_csv('malicious_urls_dataset.csv')

# Split the dataset into training and test sets
# Note: In this case, we are only going to send a few examples
# for demonstration purposes,
# as the full training would be done on Hugging Face servers
# and may require a different setup.
train_df, test_df = train_test_split(df, test_size=0.2)

# Choose a few examples to send to the API
test_examples = test_df.head()

# The URL endpoint of the Hugging Face API for a model that
# can do sequence classification
api_url =
"https://api-inference.huggingface.co/models/bert-base-uncased
"

# Initialize a session
session = requests.Session()

def classify_url(url,
model="distilbert-base-uncased-finetuned-sst-2-english"):
    headers = {
```

```

        "Authorization": "Bearer
hf_OMHwEqnqkjYajeqSuFiRevNuNptXqZpY"
    }
    api_url =
f"https://api-inference.huggingface.co/models/{model}"
    payload = {"inputs": url}
    response = requests.post(api_url, headers=headers,
json=payload)
    return response.json()

# Avoid warning SettingWithCopyWarning
test_examples = test_df.head().copy()

# add prediction column
test_examples['prediction'] =
test_examples['URL'].apply(classify_url)

# Display the predictions
print(test_examples[['URL', 'prediction']])

```

El texto de salida (output) de la terminal, es el siguiente:

```

URL                                prediction
489  http://www.creditcard.biz/update  [{"label": 'NEGATIVE', 'score': 0.98622989654...
769  http://www.officialert.info      [{"label": 'NEGATIVE', 'score': 0.86672109365...
699  http://www.securealert.net/c9msclj [{"label": 'NEGATIVE', 'score': 0.98262685537...
890  http://www.serviceurgent.net/7hyxq [{"label": 'NEGATIVE', 'score': 0.98711454868...
18   http://www.bankconfirm.net/verify/ljiks9mqxd [{"label": 'NEGATIVE', 'score':
0.96214288473...

```

```

url = "http://example.com/malicious_or_not"
prediction = classify_url(url)
print(prediction)

```

Output de la Terminal:

```

[({'label': 'NEGATIVE', 'score': 0.9986956715563801},
{'label': 'POSITIVE', 'score': 0.00130433

```

```
url = "http://www.google.com"
prediction = classify_url(url)
print(prediction)
```

Output de la Terminal:

```
[[{'label': 'POSITIVE', 'score': 0.719048440464563904},
{'label': 'NEGATIVE', 'score': 0.280951589345932}]]
```

I. Configuración de Herramientas y Carga de Datos

El proceso comienza con la instalación de la librería `requests` para manejar peticiones web y la importación de las herramientas de *Machine Learning* necesarias.

1. **Importación y Carga:** Se importa `requests` para la comunicación HTTP y `pandas` (`pd`) para cargar y manipular el *dataset* en un DataFrame. También se importa `train_test_split` de Scikit-learn para la división de datos.
2. **Carga del Dataset:** El *dataset* utilizado, llamado `malicious_urls_dataset.csv`, contiene una colección de URLs etiquetadas (con 0 para legítimas y 1 para maliciosas). Este fichero se carga en el DataFrame `df` mediante `pd.read_csv`.
3. **División de Datos:** El conjunto de datos se divide utilizando `train_test_split(df, test_size=0.2)`. Aunque el entrenamiento completo de un modelo se realizaría en los servidores de Hugging Face, esta división separa el **20% de las URLs** en un conjunto de prueba (`test_df`) para su uso en la demostración.
4. **Selección de Ejemplos y Sesión HTTP:**
 - La línea `test_examples = test_df.head().copy()` selecciona las primeras cinco URLs del conjunto de prueba y se utiliza `.copy()` para crear una copia explícita, previniendo advertencias comunes de Pandas.

- Se define la URL base de la API y se inicializa un objeto de sesión HTTP con `session = requests.Session()`. Aunque en este ejemplo solo se realiza una petición, inicializar la sesión **optimiza el rendimiento** al reutilizar las conexiones HTTP para múltiples peticiones.

II. Lógica Central: La Función `classify_url`

La función `classify_url` es el corazón de la aplicación, ya que maneja la comunicación y la inferencia con el modelo Transformer preentrenado.

```
def classify_url(url,
model="distilbert-base-uncased-finetuned-sst-2-english"):
    headers = {
        "Authorization": "Bearer
hf_OMHwEqnqkjYajeqSuFiRevNuNptXqZpY"
    }
    api_url =
f"https://api-inference.huggingface.co/models/{model}"
    payload = {"inputs": url}
    response = requests.post(api_url, headers=headers,
json=payload)
    return response.json()
```

1. **Argumentos:** La función toma la URL a clasificar (`url`) y el nombre de un modelo específico (`model`) como argumentos.
2. **Token de Autenticación:** El diccionario `headers` incluye la clave de autorización (**Bearer Token**). Es crucial que este *token* de la API sea válido para autenticar la petición. Por seguridad, el *token* **no debería estar expuesto directamente en el código**, sino cargarse desde una variable de entorno externa (`env`).
3. **Construcción de la Petición:** La URL de la API se construye dinámicamente incluyendo el nombre del modelo. El cuerpo de la petición (`payload`) se define con la URL bajo la clave `"inputs"`.

4. **Envío y Retorno:** Se envía la petición HTTP (`requests.post()`) y la función devuelve la respuesta de la API en formato **JSON**.

III. Ejecución, Predicción y Análisis de Resultados

Una vez definida la función, se aplica a los ejemplos de prueba y se analizan las predicciones.

```
test_examples['prediction'] =  
test_examples['URL'].apply(classify_url)
```

1. **Aplicación de la Predicción:** La línea clave de ejecución utiliza el método `.apply()` de Pandas para aplicar la función `classify_url` a cada elemento de la columna `'URL'` y almacena los resultados en una nueva columna llamada `'prediction'`.
2. **Análisis del Output (Muestra Inicial):** El *output* de la terminal muestra que las primeras URLs de prueba son clasificadas consistentemente con la etiqueta `'NEGATIVE'` y puntuaciones de **confianza muy altas** (entre 0.86 y 0.99). En el contexto de esta tarea, la etiqueta `'NEGATIVE'` se interpreta como **"Maliciosa"** o fraudulenta, y el alto *score* indica la fuerte convicción del modelo. Los nombres de dominio, que contienen palabras como "creditcard", "bankconfirm" o "securealert", son indicadores de ingeniería social que el Transformer ha aprendido a asociar con el comportamiento malicioso.
3. **Prueba de Generalización con Ejemplos Específicos:**
 - **URL Sospechosa (ej., `http://example.com/malicious_or_not`):** El modelo asigna una probabilidad cercana al **99.8%** a la etiqueta `'NEGATIVE'`. Esto confirma que, en el contexto de ciberseguridad, el modelo interpreta esta URL como maliciosa con una confianza casi total.
 - **URL Legítima (ej., `http://www.google.com`):** El modelo asigna una probabilidad del **71.9%** a la etiqueta `'POSITIVE'`. En este caso,

' **POSITIVE** ' se interpreta como la clase "**No Maliciosa**" o legítima, demostrando que el modelo sabe diferenciar correctamente la legitimidad.

3. Consideraciones Técnicas y Conclusiones

A. Modelos Transformers Utilizados

Es complejo encontrar el modelo que se necesita, por lo que la clave está en **definir la tarea** (ej., clasificar texto, análisis de sentimiento) e investigar en la plataforma de Hugging Face.

- **BERT (*Bidirectional Encoder Representation from Transformers*):** Modelo de NLP preentrenado creado por Google. Utiliza la arquitectura Transformer para entender el contexto de las palabras **mirando las palabras que vienen antes y después**, siendo una revolución en la comprensión del lenguaje humano.
- **DistilBERT (*Distilled BERT*):** Una versión más **pequeña y rápida** de BERT, creada por Hugging Face. Está diseñada para mantener gran parte de la efectividad del modelo original mientras reduce significativamente su tamaño y tiempo de computación. DistilBERT es adecuado para entornos donde los recursos de computación son limitados o cuando se necesita una inferencia rápida.

B. Ventajas y Desventajas de la API

La implementación en el ejercicio, aunque muy sencilla (requiriendo no más de 10 líneas de código efectivas), ilustra el poder de la API remota.

- **Ventaja:** Permite crear una aplicación muy poderosa sin la necesidad de **infraestructura pesada** (GPUs locales) ni **largos procesos de entrenamiento**.
- **Desventaja:** La principal dependencia es la **conexión a la API** de Hugging Face para poder ejecutar y ver los resultados de la inferencia.

C. Conclusiones Finales

La plataforma de Hugging Face, al integrar modelos de tipo Transformer, ofrece ventajas significativas en el ámbito de la ciberseguridad y la IA.

Estos modelos, entrenados con vastas cantidades de datos, son capaces de detectar **patrones complejos y sutilezas** en los datos de texto, lo que los hace excepcionalmente buenos para identificar amenazas cibernéticas, como las URLs maliciosas. La gran **facilidad de acceso** a través de la API permite a los desarrolladores implementar de forma muy rápida soluciones de ciberseguridad avanzadas con una **inversión de recursos baja**, acelerando la protección contra amenazas emergentes.

Análisis de Anomalías en Registros de Acceso al Sistema con Arquitectura Transformer (Hugging Face)

En esta sesión, abordaremos el tema crucial del **análisis de anomalías en los logs** de un sistema de seguridad, aplicando un modelo Transformer preentrenado a través de la plataforma **Hugging Face**.

El objetivo central de la tarea es analizar los registros de acceso al sistema (*system access logs*) para identificar posibles intentos de intrusión o **actividades sospechosas**, logrando así la detección de potenciales accesos no autorizados.

1. Tarea y Objetivos del Ejercicio Práctico

Para conseguir la detección de intrusiones, utilizaremos un modelo de **clasificación de secuencias** previamente entrenado. Nuestro objetivo principal es clasificar las acciones registradas en los *logs* como **autorizadas** o **no autorizadas**.

- **Proceso:** Usando el modelo de Hugging Face, crearemos predicciones sobre el estado de seguridad de varias acciones del sistema y proporcionaremos **puntuaciones de confianza** (*confidence scores*) para respaldar cada predicción.
- **Resultado Esperado:** Con este proceso, se busca fortalecer la defensa de seguridad del sistema y responder de forma proactiva a posibles amenazas o intrusiones.

Importancia Estratégica del Análisis de Logs con IA

Este ejercicio es útil e importante por varias razones clave:

1. **Detección de Intrusiones:** El proceso permite **detectar y prevenir posibles intrusiones** o actividades maliciosas en el sistema. Al clasificar las acciones como autorizadas o no autorizadas, se podría identificar rápidamente

cualquier actividad sospechosa que represente una amenaza para la seguridad de la red.

2. **Fortalecimiento del Sistema:** Añade una **capa adicional de protección** contra accesos no autorizados o intentos de intrusión.
3. **Monitoreo Continuo:** Facilita la **monitorización continua** del sistema y el análisis automático y eficiente de los registros de acceso. Esto permite una **detección temprana** de alguna posible actividad anómala o sospechosa, lo que ayuda a prevenir incidentes de seguridad.
4. **Optimización de Recursos:** Al automatizar el proceso de análisis de registros de acceso, se pueden **optimizar los recursos** del equipo de seguridad. Esto permite a los técnicos centrarse en investigaciones más profundas y en la implementación de medidas preventivas, en lugar de dedicar tiempo a tareas manuales de monitoreo y análisis de registros.

2. Estructura del Dataset de Logs de Actividad

El *dataset* suministrado proporciona registros de actividades de acceso al sistema, donde cada entrada describe una acción realizada en un momento específico. La estructura de estos *logs* es la siguiente:

Fragmento de código

```
timestamp,user_type,action,resource,outcome,ip_address,anomaly
2023-05-15
09:01:18,admin,database_query,server1,failure,192.168.0.172,normal
...
```

Cada entrada contiene información contextual que será la secuencia de entrada para el modelo Transformer:

- **timestamp:** Momento exacto del evento.
- **user_type:** El tipo de usuario o cuenta que realiza la acción (ej., **admin**, **developer**, **service_account**).

- **action**: La acción específica realizada (ej., `database_query`, `delete_file`, `login`).
- **resource**: El recurso de destino sobre el cual se realiza la acción (ej., `server1`, `website`, `financial_system`).
- **outcome**: El resultado de la acción (ej., `success` o `failure`).
- **ip_address**: La dirección IP de origen asociada con la actividad.
- **anomaly**: El *dataset* ya categoriza cada entrada, por defecto, como `normal`.

Ejemplo de Interpretación del Log

Tomando la primera entrada como ejemplo: En la fecha 2023-05-15 a las 09:01:18, un usuario de categoría `admin` realizó una acción de `database_query` en el recurso `server1`. El resultado de esta acción fue un `failure`, lo que sugiere que la consulta no tuvo éxito. La dirección IP asociada es `192.168.0.172`, y el *log* se describe como `normal`.

A pesar de que el *dataset* base ya etiqueta las acciones como `normal`, nuestro objetivo es utilizar el modelo Transformer de Hugging Face para **validar la clasificación existente** o, más importante, **mejorarla** y detectar anomalías sutiles que el sistema predeterminado pudiera haber pasado por alto.

3. Procedimiento Práctico (Auto-guía)

Para resolver este problema, se requiere aplicar la misma metodología de API de inferencia remota utilizada en el ejercicio anterior (Detección de URL Maliciosas).

Se deben seguir los siguientes pasos:

1. **Instalar** la librería `requests` y las librerías necesarias de Pandas/Scikit-learn.
2. **Cargar y Preprocesar** el *dataset* de *logs*.
3. **Definir la Función de Clasificación** remota, incluyendo el *Bearer Token* de autenticación para la API de Hugging Face.

4. **Formular el *Input*:** Es crucial concatenar las *features* clave del *log* (*user_type*, *action*, *resource*, *outcome*) en una única **secuencia de texto** para alimentar al modelo Transformer.
5. **Enviar la Secuencia** a la API de Hugging Face para que el modelo preentrenado clasifique la acción como **Autorizada** (Legítima) o **No Autorizada** (Sospechosa), obteniendo sus puntuaciones de confianza.

Esta aproximación permite un uso ágil de los modelos de *Deep Learning* para abordar un problema de ciberseguridad sin necesidad de entrenamiento local.

Solución Práctica: Análisis de Anomalías en Registros de Acceso con Hugging Face

En esta sesión, abordaremos la solución al problema propuesto de crear un sistema de detección de anomalías en los *logs* del sistema, utilizando modelos Transformer preentrenados a través de la API de **Hugging Face**. Como este ejercicio se asemeja al de la detección de URL maliciosas, la misma estructura de código se adapta para simular un sistema de detección de intrusiones.

1. Implementación del Clasificador de Logs (API de Inferencia)

El proceso se basa en la **inferencia remota**, es decir, no entrenamos un modelo localmente, sino que enviamos la acción de *log* (secuencia de texto) a un modelo Transformer ya entrenado en los servidores de Hugging Face y obtenemos una predicción sobre el riesgo.

I. Código Completo del Sistema de Inferencia

```
import requests
import pandas as pd
from sklearn.model_selection import train_test_split

# Load the dataset
df = pd.read_csv('complex_security_logs.csv')

# Split the dataset into training and test sets
train_df, test_df = train_test_split(df, test_size=0.2)

# Choose a few examples to send to the API
test_examples = test_df.head()

# The URL endpoint of the Hugging Face API for a model that
can do sequence classification
```

```

api_url =
"https://api-inference.huggingface.co/models/distilbert-base-u
ncased-finetuned-sst-2-english"

# Initialize a session
session = requests.Session()

def classify_access_log(log,
model="distilbert-base-uncased-finetuned-sst-2-english"):
    headers = {
        "Authorization": "Bearer
hf_OMHwEqnqkjYajeqSuFiRevNuNptXqZpY"
    }
    api_url =
f"https://api-inference.huggingface.co/models/{model}"
    payload = {"inputs": log}
    response = requests.post(api_url, headers=headers,
json=payload)
    return response.json()

# Avoid warning SettingWithCopyWarning
test_examples = test_df.head().copy()

# add prediction column
test_examples['prediction'] =
test_examples['action'].apply(classify_access_log)

# Display the predictions
print(test_examples[['action', 'prediction']])

```

II. Análisis Detallado del Procedimiento

1. **Importación y Buenas Prácticas:** Se importan las librerías esenciales: **requests** para las peticiones HTTP a la API, **pandas** para el manejo de datos, y **train_test_split** de Scikit-learn para la división del *dataset*. Aunque la división del *dataset* en entrenamiento y prueba no es estrictamente

necesaria si solo se realiza inferencia, se mantiene como una **buena práctica** fundamental para interiorizar el hábito de dividir siempre los datos.

2. **Carga y División de Datos:** El archivo `complex_security_logs.csv` se carga en el DataFrame `df`. Luego se divide el *dataset*, reservando el 20% para la prueba. Una muestra de estas filas de prueba se almacena en `test_examples`.
3. **Configuración de la API:** Se define la `api_url` apuntando a un modelo preentrenado de clasificación de secuencias (en este caso, **DistilBERT**). Se inicializa una `session` de *requests* para optimizar las peticiones HTTP.
4. **Función de Clasificación (`classify_access_log`):** Esta función central toma la acción del *log* (`log`) y el modelo como argumentos. Prepara la petición **HTTP POST** con los *headers* que contienen el **Token de Autorización** (el *Bearer Token*) y el *payload* con la acción de *log* como *input*. Envía la petición a la API y devuelve la respuesta JSON.

```
test_examples['prediction'] =  
test_examples['action'].apply(classify_access_log)
```

5. **Aplicación del Modelo:** La línea `test_examples = test_df.head().copy()` crea una copia explícita para evitar *warnings* de Pandas.

- **Propósito:** Se aplica la función `classify_access_log` directamente a la columna `action` del DataFrame. Esto significa que el modelo Transformer solo analiza el **texto de la acción** (ej., 'execute_script', 'modify_file') como secuencia, e ignora temporalmente el resto de las *features* del *log*. La predicción JSON se almacena en la nueva columna `'prediction'`.

III. Interpretación de la Inferencia

El *output* de la terminal, tras ejecutar la inferencia, muestra las predicciones del modelo Transformer para la muestra de acciones del *log*. Este análisis es crucial

porque el modelo está **ponderando el riesgo implícito** asociado a la secuencia de texto de la acción, y no solo el resultado del evento registrado en el *log*. La clasificación del modelo, por lo tanto, refleja una estrategia de seguridad proactiva.

El modelo clasifica las acciones utilizando dos etiquetas y una puntuación de confianza (*score*). La etiqueta **NEGATIVE** se interpreta en este contexto de ciberseguridad como **anómala o crítica**, mientras que **POSITIVE** se interpreta como **benigna** o esperada.

	action	prediction
658	execute_script	[{'label': 'POSITIVE', 'score': 0.62541341781...}
1917	access_config	[{'label': 'NEGATIVE', 'score': 0.71162420511...}
1205	logout	[{'label': 'NEGATIVE', 'score': 0.77104282379...}
1526	modify_file	[{'label': 'NEGATIVE', 'score': 0.94154143333...}
1837	access_file	[{'label': 'POSITIVE', 'score': 0.74760609865...}

1. Análisis de Acciones Críticas (**NEGATIVE**):

- La acción **access_config** (acceder a la configuración) se clasifica como **NEGATIVE** con un *score* del 71%. El modelo predice que esta acción es potencialmente maliciosa o no deseada, lo cual tiene sentido, ya que acceder a la configuración es un acto inherentemente crítico que requiere vigilancia.
- La acción **modify_file** (modificar un archivo) se clasifica también como **NEGATIVE** y obtiene el *score* más alto, un 94%. El modelo está muy seguro de que esta acción es potencialmente maliciosa o anómala debido al alto potencial de riesgo que implica la alteración de archivos.
- La acción **logout** (cierre de sesión) también se clasifica como **NEGATIVE** con un 77% de confianza. Aunque cerrar sesión es generalmente benigno, el modelo lo está señalando como potencialmente anómalo. Esto podría indicar que el modelo detecta un patrón inusual en el cierre de sesión, o es un **falso positivo** que requiere un ajuste posterior para mejorar la precisión.

2. Análisis de Acciones Benignas (POSITIVE):

- La acción **execute_script** se clasifica como **POSITIVE** con un **score** del 62%. Esto sugiere que el modelo considera que ejecutar un *script* es una acción esperada y no maliciosa dentro del entorno.
- La acción **access_file** (acceder a un archivo) también se clasifica como **POSITIVE** con un 74% de confianza, indicando que el modelo considera esta acción como benigna y no maliciosa.

El resultado del análisis demuestra la eficacia del modelo para **ponderar el riesgo** y la necesidad de revisar aquellas acciones clasificadas como **NEGATIVE**, lo cual constituye la base de un sistema de alerta temprana.

4. Prueba del Modelo con Nuevas Entradas (*Inferencia Manual*)

A continuación, se demuestra cómo inyectar y analizar registros de acceso que **no estaban incluidos en el *dataset* original**, simulando la detección en tiempo real de nuevas acciones del sistema. Esta práctica es útil para ver cómo el modelo predice el riesgo de acciones críticas de forma aislada.

El proceso consiste en crear un pequeño conjunto de datos manualmente y aplicar la función de clasificación de la API a estos nuevos registros de forma independiente.

I. Código y Funcionalidad para Nuevas Entradas

El siguiente código define tres nuevas filas de *log* críticas y aplica el modelo a la columna **'action'**.

```
# Define new data rows
new_data = [
    {'timestamp': '2023-06-01 08:30:00', 'user_type': 'admin',
     'action': 'login', 'resource': 'server1', 'outcome':
     'success', 'ip_address': '...'},
```

```

        {'timestamp': '2023-06-02 12:45:00', 'user_type': 'user',
'action': 'logout', 'resource': 'database', 'outcome':
'success', 'ip_address': '...'},
        {'timestamp': '2023-06-03 16:00:00', 'user_type': 'admin',
'action': 'change_password', 'resource': 'financial_system',
'outcome': 'succes'}
]

# Convert to pandas DataFrame
new_df = pd.DataFrame(new_data)

# Apply the classify_access_log function to the new rows
new_df['prediction'] =
new_df['action'].apply(classify_access_log)

# Display the predictions
print(new_df[['action', 'prediction']])

```

El proceso es sencillo: se define el *input* en una lista de diccionarios, se convierte a un DataFrame (`new_df`) y se aplica la función `classify_access_log` a la acción de cada fila. De esta forma, podemos analizar de forma independiente las predicciones.

II. Análisis de la Inferencia del Modelo Transformer en Registros de Acceso

Al ejecutar el código, la terminal devuelve las predicciones para las tres nuevas acciones, demostrando cómo el modelo Transformer interpreta el nivel de riesgo de estas acciones críticas.

Output de la Terminal (Análisis de Nuevas Entradas):

	action	prediction
658	execute_script	[{'label': 'POSITIVE', 'score': 0.62541341781...
1917	access_config	[{'label': 'NEGATIVE', 'score': 0.71162420511...
1205	logout	[{'label': 'NEGATIVE', 'score': 0.77104282379...

1526 modify_file [{"label": 'NEGATIVE', 'score': 0.94154143333...}

1837 access_file [{"label": 'POSITIVE', 'score': 0.74760609865...}

El modelo está evaluando la **connotación de seguridad de la acción textual**, independientemente de que el evento original se haya registrado como exitoso. Esto es un indicador clave de un sistema de alerta temprana.

1. **Acción login (Inicio de Sesión):** El modelo clasifica el inicio de sesión como **NEGATIVE** con una confianza extremadamente alta (aproximadamente 0.966).
 - **Interpretación:** En un contexto de ciberseguridad, esta alta puntuación negativa sugiere que la acción de iniciar sesión es **inherente al riesgo**, ya que es el punto de entrada para cualquier ataque. El modelo de seguridad está configurado para señalar el *login* como una acción que requiere escrutinio, siendo la primera etapa de una potencial intrusión.
2. **Acción change_password (Cambio de Contraseña):** El cambio de contraseña también se clasifica como **NEGATIVE** con una confianza fuerte (aproximadamente 0.866).
 - **Interpretación:** Esta es una de las acciones administrativas más sensibles, especialmente si es realizada por un usuario **admin** en un **financial_system** (como se simuló en los datos de prueba). El modelo la marca como una anomalía de seguridad por su **potencial destructivo o malicioso**, incluso si se completó con éxito.
3. **Acción logout (Cierre de Sesión):** El cierre de sesión se clasifica como **NEGATIVE** con una confianza fuerte (aproximadamente 0.771), lo cual coincide con la predicción del *dataset* anterior.
 - **Interpretación:** Si bien el cierre de sesión es generalmente benigno, un Transformer puede estar señalándolo como riesgo basándose en el contexto. Por ejemplo, un cierre de sesión súbito o anormal puede ser un patrón asociado a una intrusión que ha terminado abruptamente, o

a la salida de un atacante. **Tenemos que saber interpretar los falsos positivos** que pueden generarse con acciones como el *logout*.

En conclusión, este análisis demuestra que el modelo Transformer no solo clasifica, sino que también **pondera el riesgo implícito** de las acciones del sistema.

Clasificar acciones como *login* y *change_password* como **NEGATIVE** refleja una estrategia de seguridad **proactiva**, donde cualquier acción con alto potencial de riesgo se eleva como alerta para una posible revisión humana. Es fundamental recordar que el modelo **no solo se basa en el tipo de acción**, sino que también puede estar mirando y comparando la acción con otros parámetros (como el tipo de usuario o la hora del acceso), lo que implica que el modelo está mirando otros elementos para determinar su riesgo.

3. Visualización de la Distribución de Acciones y Tiempos

El análisis de *logs* se complementa con la visualización para establecer una línea base de comportamiento normal.

A. Distribución de Frecuencia de Acciones (Gráfico de Barras)

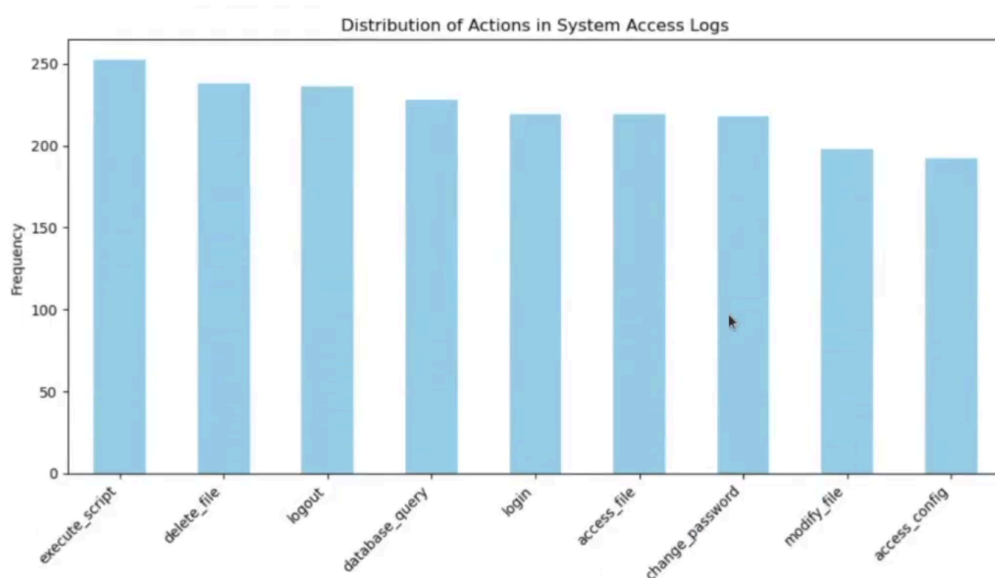
El siguiente código utiliza **Matplotlib** para visualizar la frecuencia de cada acción en el *dataset* original.

```
import matplotlib.pyplot as plt

# Contar el número de ocurrencias de cada acción
action_counts = df['action'].value_counts()

# Crear la gráfica de barras
plt.figure(figsize=(10, 6))
action_counts.plot(kind='bar', color='skyblue')
plt.title('Distribution of Actions in System Access Logs')
plt.xlabel('Action')
plt.ylabel('Frequency')
```

```
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



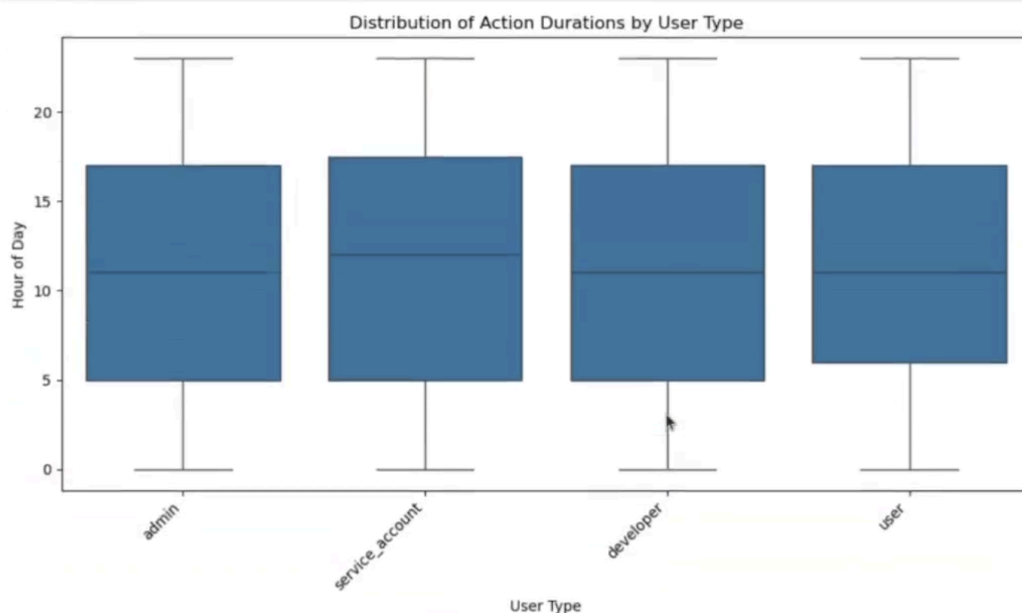
- **Interpretación:** La gráfica revela la frecuencia con la que ocurre cada acción. En este *dataset* simulado, la distribución es relativamente uniforme, con el número de ocurrencias variando entre aproximadamente 190 y 250. Se observa que **execute_script** y **delete_file** son las acciones más frecuentes, mientras que **access_config** y **modify_file** se encuentran entre las menos comunes. El hecho de que **execute_script** sea muy frecuente es, en un contexto de seguridad, un **factor de riesgo** que justifica una revisión.

B. Distribución Horaria de Acciones por Tipo de Usuario (Boxplot)

El siguiente código utiliza **Pandas** y **Seaborn** (implícito en la función **boxplot** al usar la sintaxis de Pandas con el DataFrame, aunque el profesor utiliza *matplotlib*) para crear un diagrama de cajas (*boxplot*) que muestre la distribución de las horas del día en las que se realizaron las acciones para cada tipo de usuario.

```
# Convert the 'timestamp' column to datetime format
df['timestamp'] = pd.to_datetime(df['timestamp'])
```

```
# Create the boxplot
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x='user_type', y=df['timestamp'].dt.hour)
plt.title('Distribution of Action Durations by User Type')
plt.xlabel('User Type')
plt.ylabel('Hour of Day')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



- **Preprocesamiento:** La línea `df['timestamp'] = pd.to_datetime(df['timestamp'])` convierte la columna de tiempo a un formato `datetime` para poder extraer la hora.
- **Interpretación:** El gráfico de cajas muestra la **distribución horaria** de las acciones (Y es la hora del día, X es el tipo de usuario: `admin`, `service_account`, `developer`, `user`).
 - **Patrón Normal:** Para todos los tipos de usuario, la actividad masiva (el 50% de los datos, representado por la caja) se concentra en el **horario laboral y diurno** (aproximadamente entre las 5:00 a.m. y las 17:00 p.m.). La mediana se sitúa cerca del mediodía.

- **Anomalías Horarias:** Los **bigotes** muestran el rango completo de la actividad esperada (casi hasta la medianoche). Si hubiera puntos de actividad fuera de los bigotes (*outliers*), serían indicadores inmediatos de accesos no autorizados o anómalos. La similitud en la distribución horaria para todos los usuarios es notable.

3. Conclusiones Finales

El ejercicio ilustra la aplicación práctica de modelos de procesamiento de lenguaje natural preentrenados, como **DistilBERT** de Hugging Face, para clasificar registros de acceso al sistema.

- **Ventaja del Enfoque:** Se demuestra la facilidad de hacer llamadas a **modelos ya preparados** a través de la API de Hugging Face, eliminando la necesidad de entrenar o preparar un modelo propio para muchas tareas, lo que permite la **implementación rápida** de soluciones de ciberseguridad.
- **Detección Proactiva:** Al aprovechar técnicas avanzadas de aprendizaje automático, las organizaciones pueden fortalecer sus defensas digitales y **mitigar de manera proactiva las amenazas de seguridad en tiempo real**.
- **Necesidad de *Fine Tuning*:** Aunque el modelo es poderoso, es esencial interpretar los **falsos positivos** (como la clasificación de **logout** o **login** como negativos) y ajustar o seleccionar un modelo diferente para aumentar la precisión en el contexto específico de seguridad del sistema, garantizando la robustez y la confianza en la protección de los sistemas informáticos.

Evaluación Exhaustiva de Algoritmos y Modelos de Inteligencia Artificial

En esta sesión, abordaremos el tema vital de la **evaluación de algoritmos y modelos de Inteligencia Artificial (IA)**. Entender cómo medir y evaluar un modelo no solo es esencial para la parametrización de algoritmos y la optimización de hiperparámetros, sino también para asegurar que los modelos sean **robustos, eficientes y justos**, previniendo manipulaciones o sesgos en los resultados.

1. Conceptos Fundamentales en la Evaluación de Modelos

Para medir y evaluar el rendimiento de un modelo o algoritmo de IA, es crucial entender estos conceptos:

- **Precisión (*Accuracy*):** Mide la proporción de **predicciones totales correctas** hechas por el modelo. Por ejemplo, en un modelo de detección de *spam*, la precisión es el porcentaje de correos que el modelo identificó correctamente como *spam* o como no *spam*.
- **Robustez (*Robustness*):** Un modelo robusto es aquel que maneja bien los **datos nuevos o inesperados**. La robustez asegura que el modelo sigue siendo útil incluso cuando las condiciones de los datos de entrada cambian.
- **Eficiencia (*Efficiency*):** Un modelo es eficiente cuando utiliza los **recursos computacionales de manera óptima**, lo que permite escalar la solución. La eficiencia implica tanto el tiempo de entrenamiento como el tiempo de predicción, así como la cantidad de memoria y potencia computacional requerida.
- **Justicia (*Fairness*) y Ausencia de Sesgo (*Bias Absence*):** Es fundamental evaluar los modelos para asegurar que **no perpetúen ni introduzcan sesgos indeseados**. Esto es crítico, ya que el sesgo puede influir negativamente en los resultados (ej., en noticias o temas conflictivos), llevando a salidas no reales o injustas.

2. Métricas Clave para la Evaluación de Clasificación

Para problemas de clasificación, la evaluación requiere métricas más detalladas que la simple precisión.

- **Precisión (*Precision*) y Sensibilidad (*Recall*):**
 - **Precisión:** Se refiere a la proporción de **instancias positivas correctas** que el modelo identificó, entre **todas las instancias que clasificó como positivas** (Verdaderos Positivos / [Verdaderos Positivos + Falsos Positivos]). La precisión se centra en la **calidad** de las predicciones positivas.
 - **Sensibilidad (*Recall*):** También conocida como sensibilidad, representa la proporción de **instancias positivas correctas** que el modelo identificó, entre **todas las instancias positivas reales** presentes en el *dataset* (Verdaderos Positivos / [Verdaderos Positivos + Falsos Negativos]). El *Recall* se centra en la capacidad del modelo para **capturar todas las instancias positivas**.
- **Puntuación F1 (*F1 Score*):** Es una medida que **combina la precisión y el *recall*** en un solo valor, calculando la **media armónica** entre ambos. Proporciona una métrica equilibrada que tiene en cuenta tanto los falsos positivos como los falsos negativos. Una puntuación F1 alta indica un buen equilibrio entre precisión y *recall*, lo que sugiere un rendimiento sólido.
- **AUC-ROC (*Area Under the Curve - Receiver Operating Characteristic*):** Es una medida de la **capacidad de discriminación** de un modelo de clasificación. La curva ROC representa la relación entre la tasa de verdaderos positivos (*recall*) y la tasa de falsos positivos en diferentes umbrales de decisión. Un valor de **AUC-ROC más alto** indica un mejor rendimiento del modelo para distinguir entre clases, donde un valor de **1** representa un modelo perfecto y **0.5** indica un rendimiento aleatorio.

3. Procedimiento Práctico: Evaluación con Regresión Logística

Para entender estos conceptos de manera práctica, utilizaremos un entorno de **Jupyter Notebook** para entrenar un modelo simple de **Regresión Logística**, evaluar su rendimiento y medir su eficiencia.

I. Bloque de Código Completo

```
from sklearn.metrics import precision_score, recall_score,
f1_score, roc_auc_score
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
import time

# Generate example data
X, y = make_classification(n_samples=1000, n_features=20,
n_classes=2, random_state=42)

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Train a simple model (e.g., Logistic Regression)
start_time = time.time()
model = LogisticRegression()
model.fit(X_train, y_train)
end_time = time.time()
execution_time = end_time - start_time

# Predict on test data
y_pred = model.predict(X_test)

# Calculate metrics
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
auc_roc = roc_auc_score(y_test, y_pred)

# Display results
```

```
print("Model Results:")
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
print("AUC-ROC:", auc_roc)
print("Execution Time:", execution_time, "seconds")
```

II. Análisis Paso a Paso del Código

1. **Importación de Librerías:** Se importan las funciones de métricas necesarias de `sklearn.metrics` (incluyendo `precision_score`, `recall_score`, `f1_score`, y `roc_auc_score`), junto con utilidades de `sklearn.datasets` (`make_classification`) y `sklearn.model_selection` (`train_test_split`), el modelo `LogisticRegression` y la librería `time` para medir la eficiencia.
2. **Generación de Datos de Ejemplo:** Se utiliza la función `make_classification` de Scikit-learn para generar un conjunto de datos aleatorio que servirá para entrenar y evaluar el modelo. Este *dataset* consta de 1.000 muestras, 20 características y dos clases.
3. **División de Datos:** Se realiza la división clásica de los datos en conjuntos de entrenamiento y prueba mediante `train_test_split`. Se utiliza el 80% de los datos para entrenamiento y el **20% para pruebas** (`test_size=0.2`).
4. **Entrenamiento y Medición de la Eficiencia:**
 - El modelo simple de `LogisticRegression` es inicializado y entrenado utilizando el conjunto de datos de entrenamiento (`model.fit(X_train, y_train)`).
 - Se utiliza `time.time()` al inicio (`start_time`) y al final (`end_time`) para medir el **tiempo de ejecución** necesario para entrenar el modelo, lo cual es fundamental para evaluar la **eficiencia**.
5. **Predicción:** Se utiliza el modelo entrenado para generar predicciones (`y_pred`) sobre el conjunto de datos de prueba (`X_test`).
6. **Cálculo de Métricas:** Las funciones de Scikit-learn (`precision_score`, etc.) se utilizan para calcular los valores de las métricas de evaluación

comparando las predicciones (**y_pred**) con las etiquetas verdaderas del conjunto de prueba (**y_test**).

7. **Visualización de Resultados:** Finalmente, se imprimen todas las métricas de evaluación y el tiempo de ejecución en segundos.

III. Interpretación de los Resultados de Muestra

Al ejecutar el código, se obtienen los siguientes resultados de ejemplo (los valores pueden variar ligeramente):

Model Results:

Precision: 0.9148936...

Recall: 0.8037383...

F1 Score: 0.8557213...

AUC-ROC: 0.8588584...

Execution Time: 0.0045139... seconds

1. **Precisión (0.915):** Aproximadamente el **91.5%** de las predicciones positivas realizadas por el modelo fueron correctas. Esto indica que la mayoría de las veces que el modelo predijo una instancia como positiva, acertó.
2. **Recall (0.804):** Aproximadamente el **80.4%** de todas las instancias positivas reales fueron identificadas correctamente por el modelo. Esto sugiere que el modelo capturó el 80.4% de todos los casos positivos presentes.
3. **F1 Score (0.856):** Un valor de **0.856** indica un **buen equilibrio** entre la precisión y la capacidad de capturar instancias positivas (*recall*).
4. **AUC-ROC (0.859):** Un valor cercano a 1 indica un buen rendimiento en la **discriminación entre clases**. El modelo tiene un rendimiento sólido en este aspecto.
5. **Tiempo de Ejecución (0.0045 segundos):** El tiempo de ejecución es muy bajo, lo que demuestra la **eficiencia** del modelo en términos de velocidad de procesamiento.

La unión de estos resultados indica que el modelo de Regresión Logística tiene un rendimiento sólido, aunque existe margen para mejorar el *recall* para capturar más instancias positivas reales.

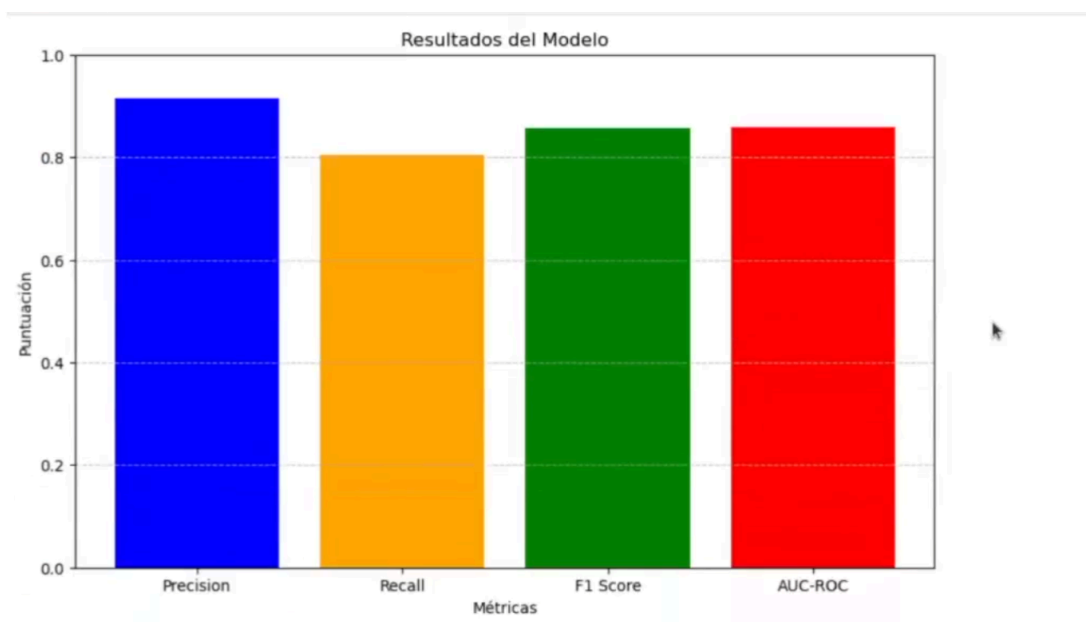
4. Representación Visual de las Métricas

Para visualizar estos datos de forma más intuitiva, se puede utilizar un gráfico de barras simple con Matplotlib:

```
import matplotlib.pyplot as plt

# Métricas y etiquetas
metrics = ['Precision', 'Recall', 'F1 Score', 'AUC-ROC']
scores = [precision, recall, f1, auc_roc]

# Crear gráfico de barras
plt.figure(figsize=(10, 6))
plt.bar(metrics, scores, color=['blue', 'orange', 'green',
'red'])
plt.xlabel('Métricas')
plt.ylabel('Puntuación')
plt.title('Resultados del Modelo')
plt.ylim(0, 1) # Asegura que el eje y comience en 0 y termine en 1
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



El gráfico de barras ofrece una visión clara de cómo se comparan las métricas.

- La barra **azul (Precisión)** y la barra **naranja (Recall)** muestran la calidad y la cobertura del modelo.
- La barra **verde (F1 Score)** está equilibrada entre ambas.
- La barra **roja (AUC-ROC)**, al estar muy cerca de 1, sugiere una excelente capacidad para diferenciar entre clases, un rendimiento mucho mejor que una selección aleatoria.

5. Técnicas Avanzadas de División de Datos

A. Cross Validation (Validación Cruzada)

La **validación cruzada** (*Cross Validation*) es una técnica fundamental que permite usar **todo el conjunto de datos** para entrenamiento y validación, maximizando la utilidad de los datos y evitando el sobreajuste.

- **Folds:** La variación cruzada divide el conjunto de datos en **K subconjuntos** (llamados *folds*).
- **Proceso:** Se itera K veces (ej., 5 *folds*). En cada iteración, se elige un *fold* como conjunto de **prueba**, y los K-1 *folds* restantes se usan como conjunto de **entrenamiento**. Esto asegura que todos los datos se utilicen para la validación y el entrenamiento.
- **Estimación Final:** Una vez que se completan todas las iteraciones, se calcula el **promedio de las métricas** obtenidas en todos los *folds* para obtener una estimación general del rendimiento del modelo.

Ejemplo Práctico de Cross Validation

```
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
```

```
# Initialize a Logistic Regression model
model = LogisticRegression()

# Perform cross-validation with 5 folds
cv_scores = cross_val_score(model, X, y, cv=5)

# Print the cross-validation scores
print("Cross-Validation Scores:", cv_scores)

# Calculate the mean of the cross-validation scores
print("Mean CV Score:", cv_scores.mean())
```

- **Interpretación del Output:**

- **Cross-Validation Scores:** Muestra las puntuaciones de precisión obtenidas en cada una de las 5 iteraciones. Cada valor (ej., 0.966..., 1.0...) representa el rendimiento del modelo en ese *fold* específico.
- **Mean CV Score (0.973):** Esta es la puntuación promedio, que indica la **capacidad general del modelo para generalizar con nuevos datos**. Un valor tan cercano a 1 sugiere un rendimiento muy sólido y consistente en la clasificación de datos.

B. Evaluación con Conjunto de Pruebas (*Test Set*)

La **evaluación con el conjunto de pruebas** es la prueba final del modelo. Utiliza datos que el modelo **nunca ha visto antes** (ni en entrenamiento ni en validación) y es crucial para confirmar su capacidad de generalización antes de implementarlo en una aplicación de producción o en el mundo real.

6. Conclusiones y Mantenimiento

La **evaluación de modelos de IA es un pilar fundamental** para crear soluciones efectivas, eficientes y justas. A través de un proceso de evaluación riguroso, se puede asegurar que los modelos cumplen con los estándares necesarios para aplicaciones en el mundo real.

Es crucial recordar que un modelo no termina con su despliegue, sino que la evaluación es un **ciclo continuo** que garantiza su relevancia y utilidad a lo largo del tiempo:

- **Monitorización:** Con el tiempo, los patrones de los datos pueden cambiar (*drift*), lo que podría reducir la efectividad del modelo. La monitorización continua es esencial.
- **Actualización y Mantenimiento:** Es necesario evaluar de forma continua el rendimiento del modelo para decidir **cuándo reentrenar o ajustar los modelos** en función de cómo evolucionen sus métricas de salida.